

Jour 9 : Retransmission et Robustesse

1. Définition des Concepts Clés

- **ARQ (Automatic Repeat reQuest)** : Mécanisme de contrôle d'erreurs où l'émetteur retransmet automatiquement un paquet s'il n'a pas reçu d'accusé de réception (ACK) pour celui-ci.
- **RTO (Retransmission TimeOut)** : Le délai d'attente avant de décider qu'un paquet est perdu. Si $RTO = 3s$ et que je n'ai pas de réponse après 3s, je considère le paquet perdu.
- **Congestion Collapse** : Situation catastrophique où le réseau est saturé non pas par des données utiles, mais par des *copies* de données retransmises inutilement à cause de délais mal calibrés.

2. Utilité : Pourquoi la Retransmission ?

La couche IP (Internet Protocol) est "**Best Effort**" : elle ne garantit rien. Un routeur peut jeter votre paquet s'il est plein. TCP a pour mission de fournir un **service fiable** sur ce réseau non fiable.

- **Fiabilité** : Assurer que 100% des octets arrivent, et dans le bon ordre.
- **Robustesse** : Ne pas planter si un câble est débranché temporairement.

3. Focus Cyber : Attaques sur la Congestion

Si chaque pile TCP renvoyait ses paquets immédiatement après une perte sans ralentir, le réseau entrerait dans un cercle vicieux de retransmissions massives, stoppant tout trafic utile.

1. L'Attaque "Shrew" (L'attaque du Musaraigne)

C'est une attaque **DoS à bas débit** redoutable. L'attaquant envoie des pics de trafic très brefs mais intenses à des intervalles réguliers calqués sur le **RTO** standard (généralement 1 seconde ou 3 secondes).

- **Mécanisme** : La victime subit une perte lors du pic, attend son RTO, tente de renvoyer... et paf, un nouveau pic arrive pile au moment de sa retransmission.
- **Résultat** : Le débit de la victime tombe à zéro alors que l'attaquant consomme très peu de bande passante.
- **Défense** : Notre implémentation du **Backoff Exponentiel** et l'ajout (théorique) de **Jitter** (variation aléatoire du timer) cassent cette synchronisation malveillante.

2. Manipulation des ACKs (Optimistic ACK Attack)

Un attaquant peut envoyer des ACKs pour des données qu'il n'a pas encore reçues. Cela "trompe" notre pile en lui faisant croire que le réseau est ultra-rapide, poussant notre cwnd à augmenter artificiellement.

- **Risque** : Cela force notre serveur à envoyer des données à une vitesse folle, provoquant sa propre congestion ou saturant son lien de sortie.

Le mécanisme de retransmission peut être retourné contre le serveur.

- **Low-Rate TCP DoS (Shrew Attack)** : Un attaquant envoie des rafales périodiques pour provoquer des pertes juste au moment où le serveur allait retransmettre, le forçant à attendre encore plus longtemps (backoff exponentiel), réduisant son débit à zéro.
- **Optimistic ACK Attack** : Un attaquant envoie des ACKs pour des données qu'il n'a pas encore reçues, trompant le serveur qui va inonder le réseau, provoquant un déni de service pour les autres.

4. Partie du Code Essentielle (tcp.c)

L'implémentation repose sur le suivi du temps (last_send_time) dans le TCB.

```
// Fonction appelée à chaque tour de boucle dans main.c
void tcp_check_timeouts(int fd) {
    // Si on est en attente d'ACK (ex: SYN_RECEIVED)
    if (conn->state == TCP_SYN_RECEIVED) {
        // Et que le délai est dépassé (RTO = 3s)
        if (difftime(now, conn->last_send_time) >= TCP_RTO) {

            // On limite le nombre d'essais pour éviter les boucles infinies
            if (conn->retransmit_count >= MAX_RETRY) {
                conn->state = TCP_CLOSED; // Abandon
                return;
            }
            printf("TCP TIMEOUT: Retransmission SYN-ACK...\n");
            // ... (Code de reconstruction et renvoi du paquet) ...

            conn->last_send_time = now; // On reset le timer
            conn->retransmit_count++;
        }
    }
}
```

5. Protocole de Test

Nous allons simuler un client "muet" qui initie une connexion (SYN) mais refuse de répondre à notre SYN-ACK.

Commande :

1. Lancez la stack : `sudo ./arp_stack`
2. Lancez un scan SYN (qui ne complète pas le handshake) : `sudo nmap -sS -p 80 10.0.0.2`

6. Résultats Attendus

Les logs doivent montrer la séquence suivante :

1. TCP: Requête de connexion SYN de ... (Réception initiale)
2. TCP: SYN-ACK envoyé (Réponse normale)
3. *(Silence de 3 secondes)*
4. TCP TIMEOUT: Retransmission SYN-ACK pour TCB 0 (Essai 1/5)
5. *(Silence)*
6. TCP TIMEOUT: Retransmission SYN-ACK pour TCB 0 (Essai 2/5) ...
7. TCP: Trop de retransmissions... Connexion fermée.

7. Interprétation

- **Succès** : Si vous voyez les messages "Retransmission", votre stack est capable de détecter une perte (simulée ici par l'absence de réponse) et de réagir.
- **Échec** : Si le log s'arrête après "SYN-ACK envoyé" et ne bouge plus, votre stack est "bloquante" et vulnérable aux pannes réseaux.